

Technische Informatik

Versionskontrollsysteme und Git

AUTOR

Prof. Johannes Baumgart  

ZUGEHÖRIGKEIT

Hochschule Esslingen

VERÖFFENTLICHT

19. September 2024

1 Versionskontrolle mit git



Hast du mal die Version von Gestern? Grafik von Jugaad Posters

„Gestern gings noch ... wo ist nur die Version von Gestern.“

1.1 Lernziele

- Kennen Sie die Vor- und Nachteile von Versionsverwaltungssystemen
- Verständnis über die verschiedenen Konzepte von Versionsverwaltungssystemen
- Operatives verwenden von git zur Versionierung und eines lokalen Repositories
- Operatives verwenden von Branches in einem Branching Model
- Grundlagen von Code Reviews verstehen und diese anhand der Checkliste durchführen können

1.2 Grundlagen Versionskontrollsysteme ¹

Versionskontrollsystem (dt. Versionskontrollsystem (VCS))

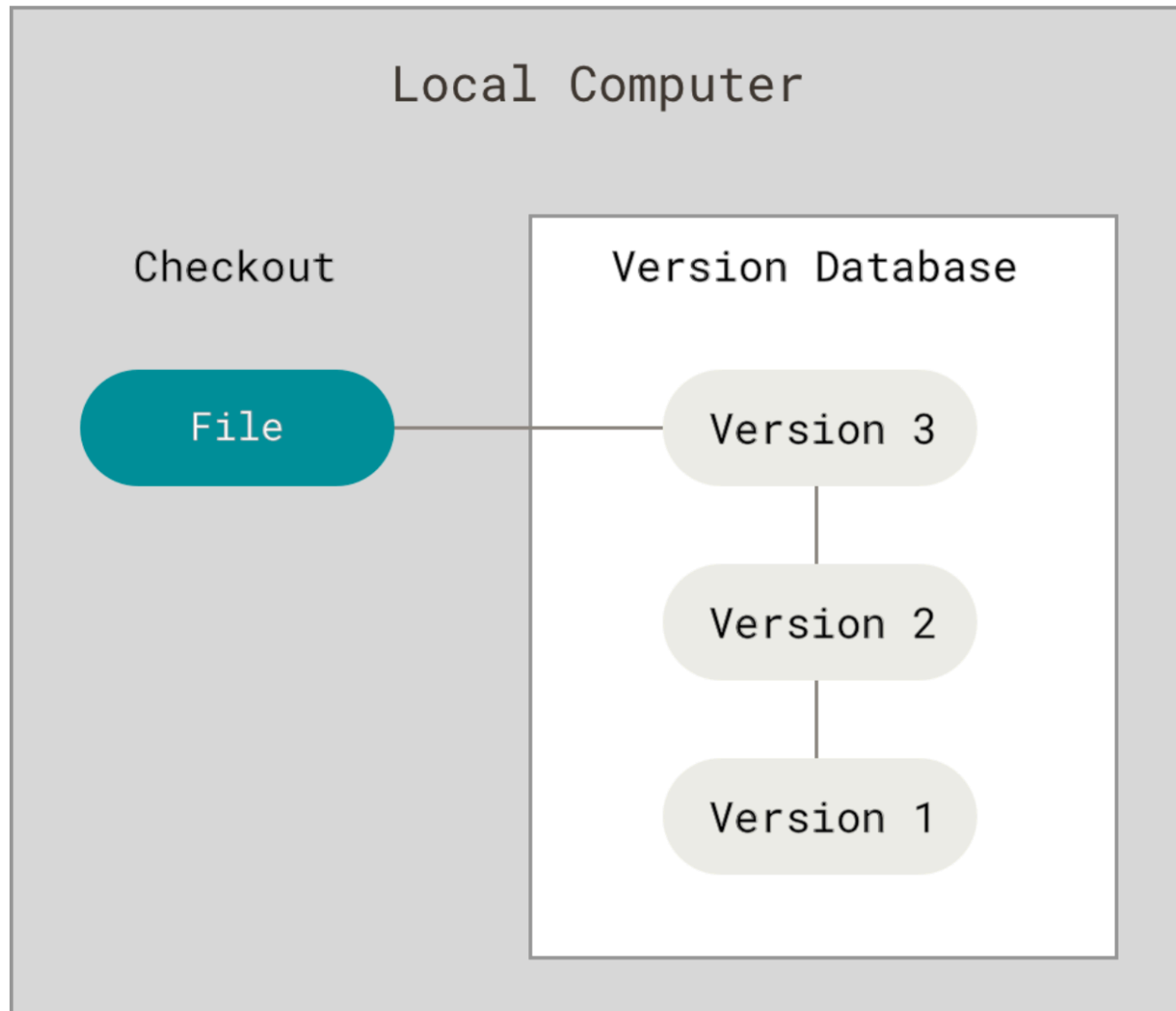
Das Versionskontrollsystem ist ein System zur Aufzeichnung von Änderungen an einer Datei oder einem Satz von Dateien im Laufe der Zeit aufzeichnet, damit Sie bestimmte Versionen später wieder aufrufen können.

Generell ermöglicht ein Versionskontrollsystem es einem, wenn man einen Fehler gemacht hat, auf eine frühere Version einer Datei zurückzuspringen.

Esg gibt 3 prinzipielle Ansätze:

- Lokale Versionskontrolle
- Zentrale Versionskontrolle
- Verteilte Versionskontrolle

1.2.1 Lokale Versionskontrolle



Lokale Versionsverwaltung

Ist die erste Variante von VCS. In der eine einfache lokale Datenbank zum Einsatz kommt, die die Veränderungen einer kontrollierten Datei in einzelnen „Versionen“ speichert.

Hier ist das ursprüngliche Problem der „Umbenennungsversionierung“ gelöst. Aber noch keine Zusammenarbeit möglich.

Projektarchiv

Als Repository wird eine Datenbank mit verschiedenen Versionsständen, Kommentaren, Tags etc. verstanden.

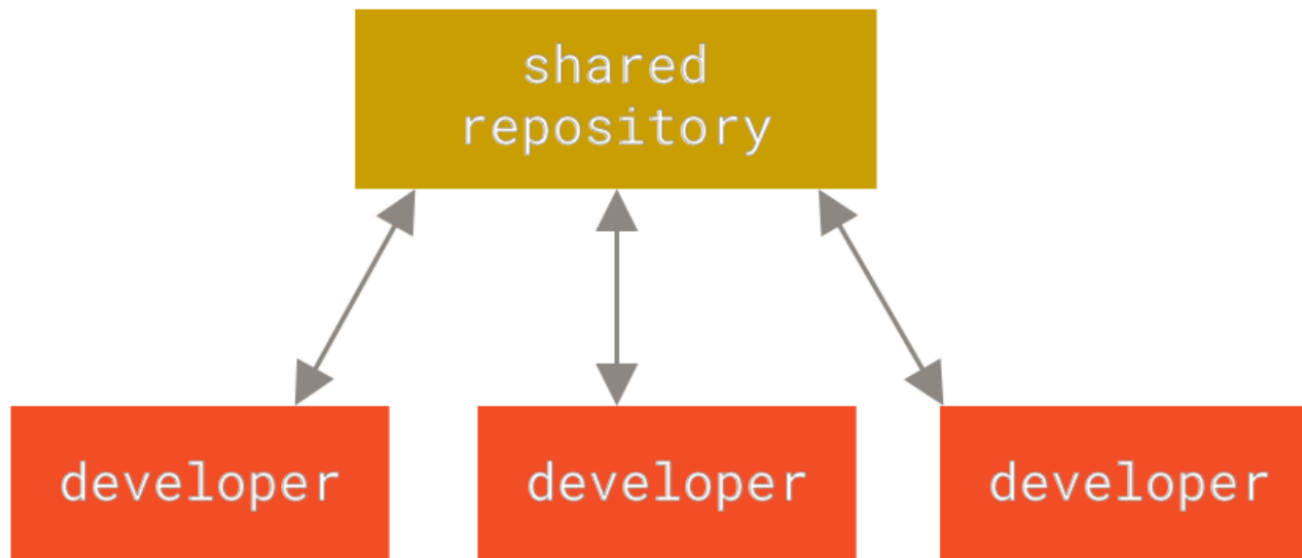
Arbeitskopie

Als Arbeitskopie wird die Arbeitskopie in einem bestimmten Stand verstanden.

1.2.2 Zentrale Versionskontrolle

Um zu ermöglichen, dass verschiedene Menschen miteinander arbeiten können, sind die zentralisierten VCS entstanden (zB CVS, Subversion).

Ein Server beinhaltet alle versionierten Dateien. Die Clients überprüfen die Dateien von diesem zentralen Ort aus, ohne deren Versionierung.



Zentrale Versionskontrolle

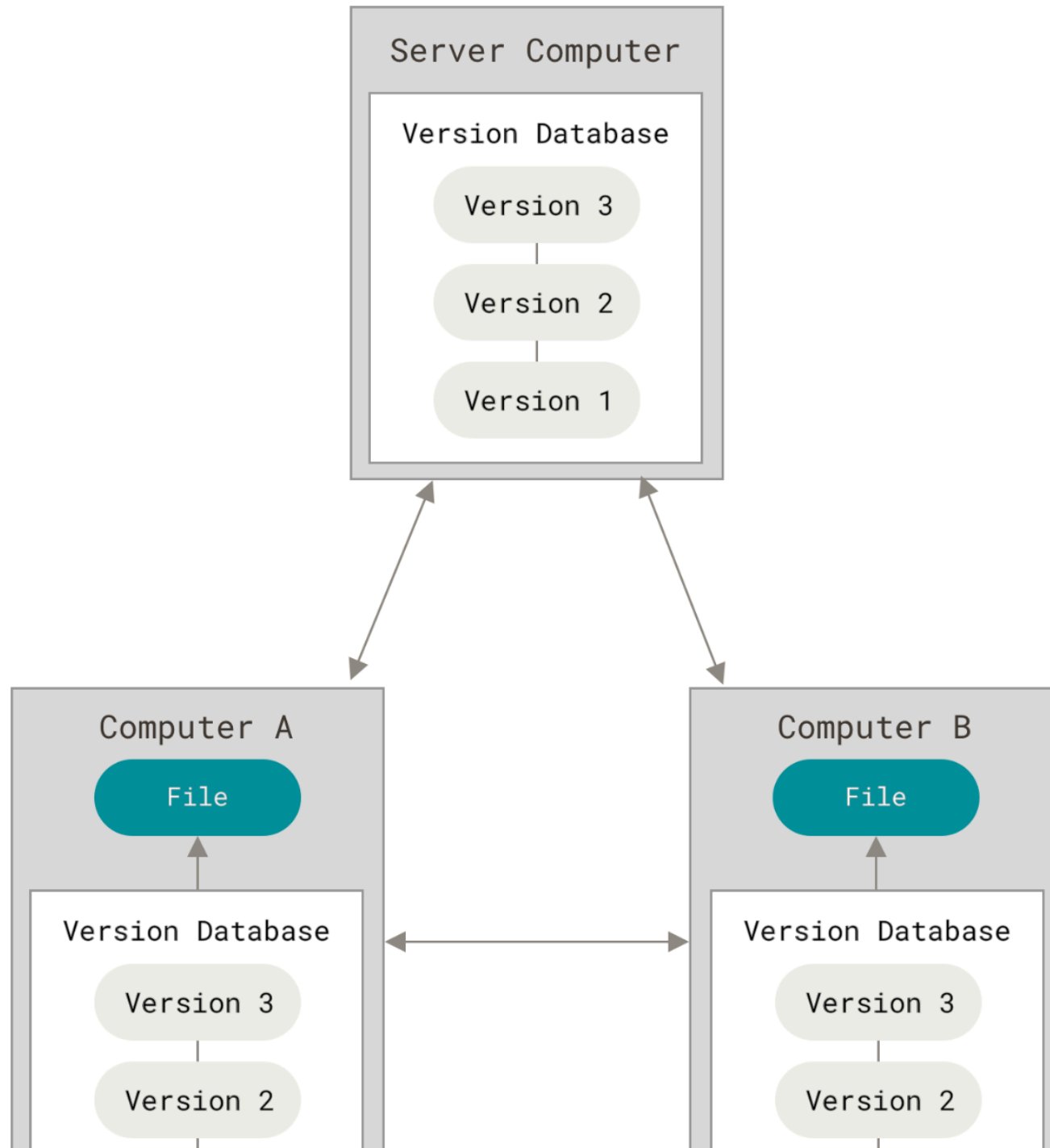
Diese Systeme waren lange Zeit der Standard. Problematisch war, dass es nur einen Ort gibt auf dem die Historie gespeichert ist.

- Verfügbarkeit
- Beschäftigung
- Einzelner Ausfallpunkt

1.2.3 Verteilte Versionskontrolle

Bei verteilten VCS ist es nicht nur so, dass die Dateien auf den Clients ausgecheckt werden, sondern auch die Historie.

Das behebt die grundsätzlichen Probleme von zentralem VCS. Das Zusammenführen unterschiedlicher lokaler Versionen und Ständen ist jedoch deutlich komplizierter. Bekannte Vertreter dieser VCS sind: git und Mercurial.





Teile Versionskontrolle

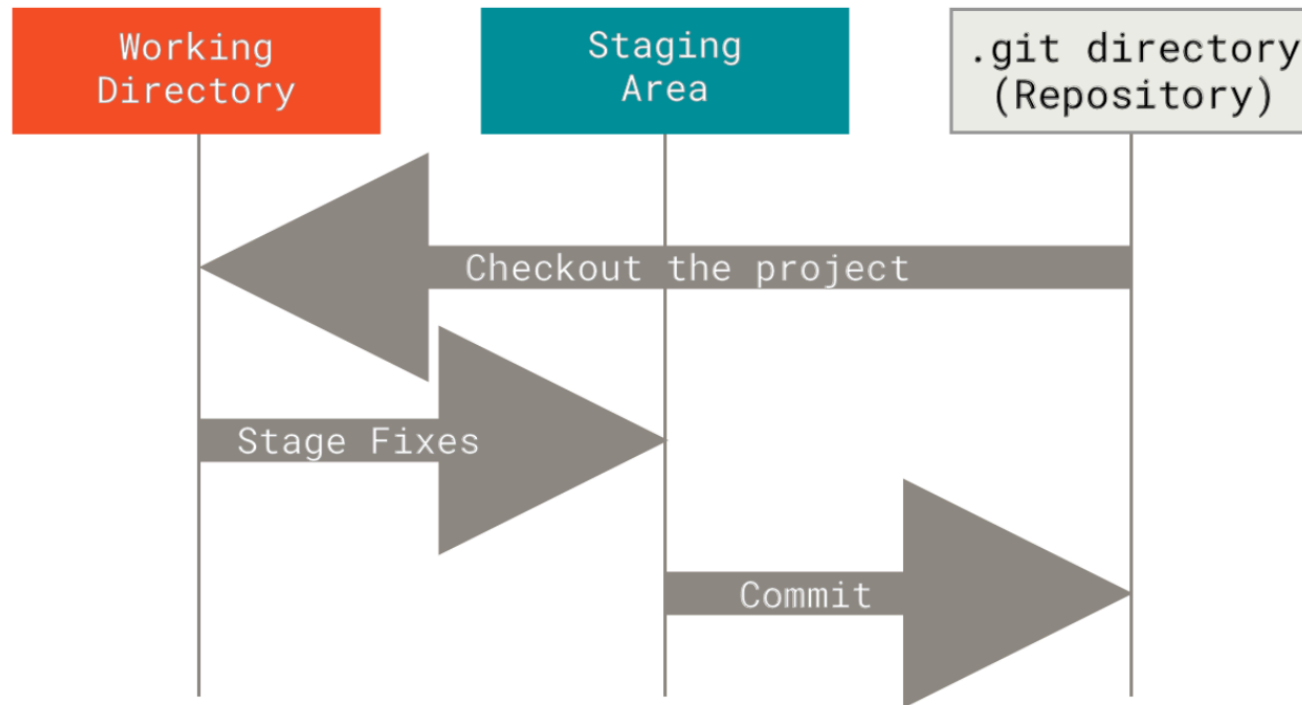
Fast jede Veränderung wird im lokalen Repository vorgenommen. Zentral wird nur noch auf Versionen der unterschiedlichen lokalen Repositories betrieben.

1.3 Versionskontrollsystem Git

Zusammenfassung

Git kennt drei grundsätzliche Bereiche:

1. Arbeitsverzeichnis
2. Bereitstellungsbereich
3. Projektarchiv

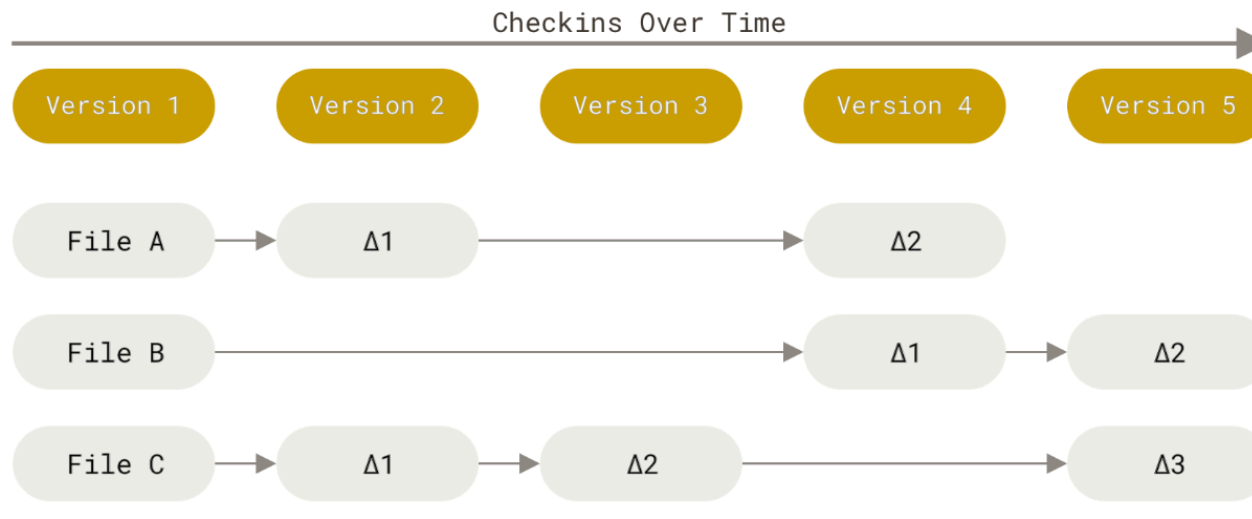


Konzeptionelle Bereiche in git mit deren Zustandsübergängen

Die Staging Area ist ein neues Konzept. In ihr befinden sich Änderungen, die in einem gemeinsamen Commit in das Repository übertragen werden sollen.

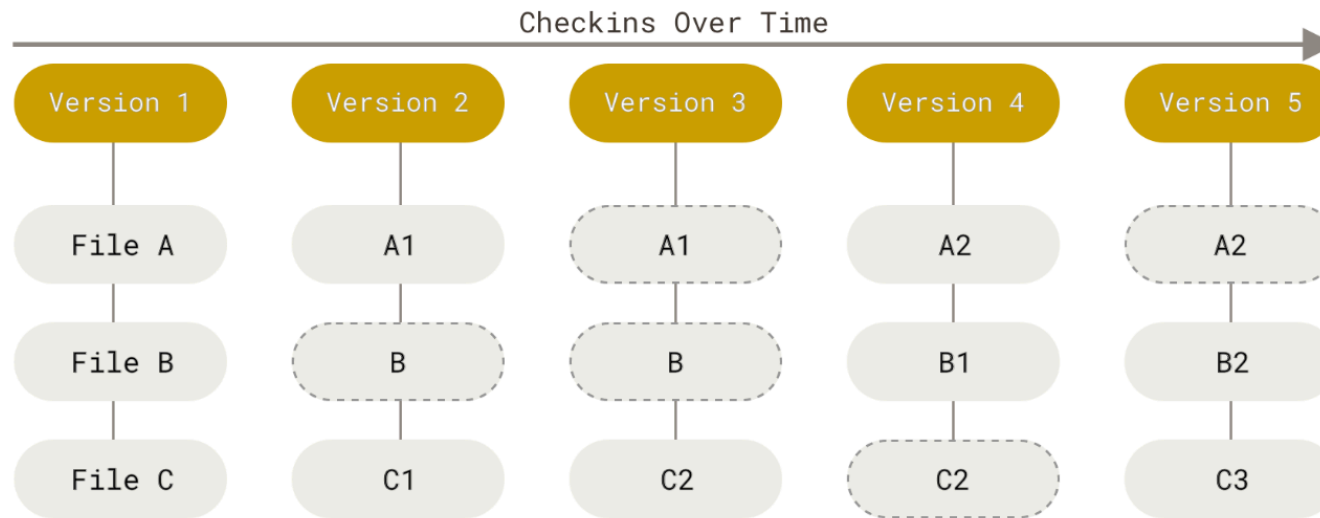
1.3.1 Was ist in einer Version?

Der Hauptunterschied zwischen Git und jedem anderen VCS (einschließlich Subversion und Konsorten) ist die Art und Weise, wie Git-Versionen verwaltet werden. Vom Konzept her speichern die meisten anderen Systeme Informationen als eine Liste von dateibasierten Änderungen. Diese anderen Systeme (CVS, Subversion, Perforce usw.) betrachten die Informationen, die sie speichern, als eine Reihe von Dateien und die Änderungen, die an jeder Datei im Laufe der Zeit vorgenommen wurden (dies wird allgemein als delta-basierte Versionskontrolle bezeichnet) .



Delta Versionierung eines VCS

Git speichert seine Daten nicht auf diese Weise. Stattdessen betrachtet Git seine Daten eher als eine Reihe von Schnappschüssen (eng. Snapshots) eines Miniatur-Dateisystems. Jedes Mal, wenn Sie mit Git einen Commit ausführen oder den Status des Projekts speichern, macht Git im Grunde ein Bild davon, wie alle Dateien in diesem Moment aussehen, und speichert einen Verweis auf diesen Schnappschuss. Um effizient zu sein, speichert Git die Datei nicht erneut, wenn sie sich nicht geändert hat, sondern nur einen Verweis auf die vorherige identische Datei, die bereits gespeichert wurde. Git betrachtet seine Daten eher als einen Strom von Schnappschüssen.



Git-Versionierung: Jede Version beinhaltet einen Snapshot des Mini-Dateisystems.

Alles in Git wird mit einer Prüfsumme versehen, bevor es gespeichert wird, und dann wird auf diese Prüfsumme verwiesen. Dies bedeutet, dass es **unmöglich** ist, den Inhalt einer Datei oder eines Verzeichnisses zu ändern, ohne dass Git davon erfährt. Diese Funktionalität ist ein wesentlicher Bestandteil der Philosophie von Git. Es können also keine Informationen verloren gehen oder Dateien beschädigt werden, ohne dass Git dies erkennen kann.

Die Prüfsumme ist ein SHA-1-Hash. zB:

24b9da6552252987aa493b52f8696cd6d3b00373

Git sorgt durch diese Prüfsummen für Datenintegrität, da jede Änderung zu einer Veränderung der Prüfsumme führt und jedes Commit mit notwendigen Attributen wie eMail und Name versehen ist.

Datenintegrität ²

Integrität bezeichnet die Sicherstellung der **Korrektheit (Unversehrtheit)** von Daten und der korrekten Funktionsweise von Systemen. Wenn der Begriff Integrität auf „Daten“ angewendet wird, drückt er aus, dass die Daten **vollständig und unverändert** sind. In der Informationstechnik wird er in der Regel aber weiter gefasst und auf „Informationen“ angewendet. Der Begriff „Information“ wird dabei für „Daten“ verwendet, denen je nach Zusammenhang **bestimmte Attribute** wie z. B. Autor oder Zeitpunkt der Erstellung **zugeordnet werden**

können. Der **Verlust der Integrität** von Informationen kann daher bedeuten, dass diese **unerlaubt verändert**, Angaben zum Autor verfälscht oder Zeitangaben zur Erstellung manipuliert wurden.

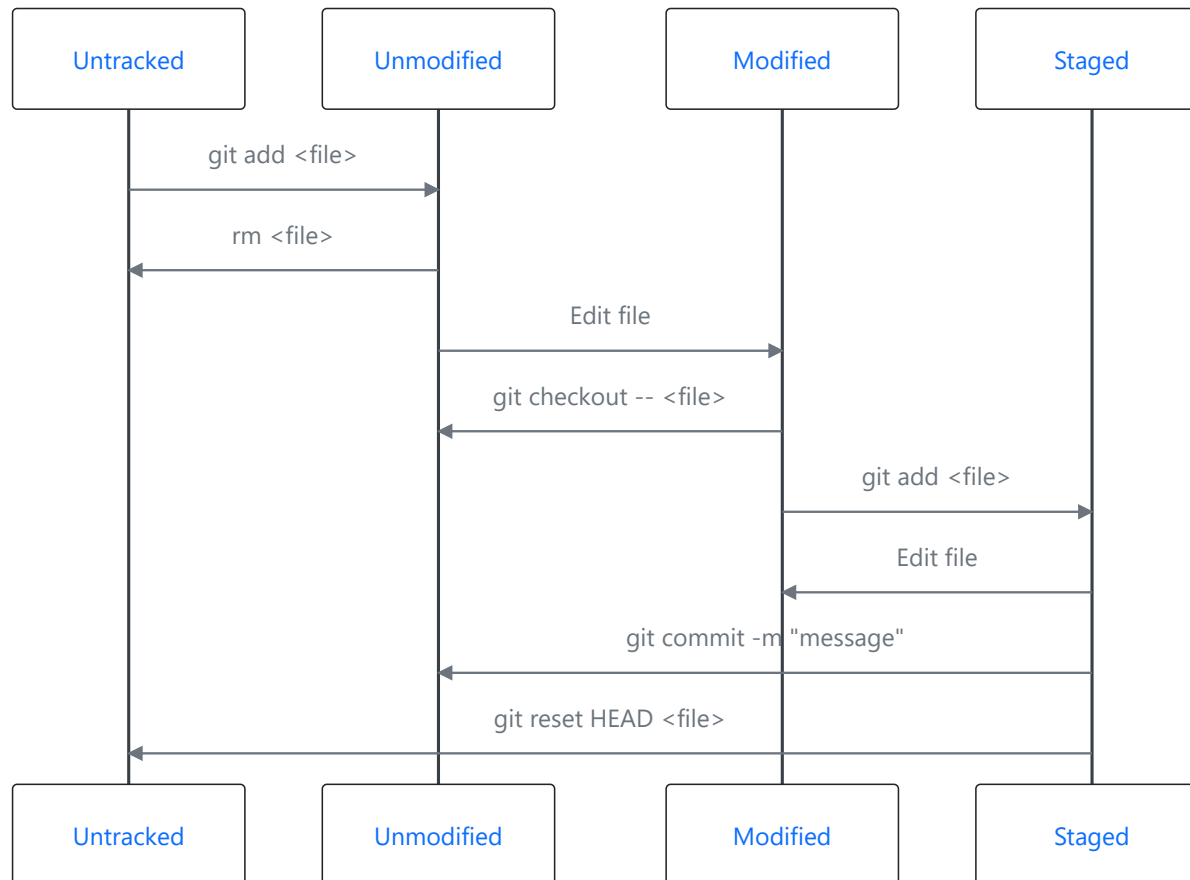
1.3.2 Die drei Zustände

Das elementare Konzept von Git basiert auf drei Zuständen die eine Datei einnehmen kann. Diese sind:

- **modified:** Die Datei hat sich geändert. Die Änderung ist aber noch nicht in der Datenbank.
- **staged:** Die Änderungen sind vorgemerkt, dass sie mit dem nächsten commit in der Datenbank gespeichert werden.
- **committed:** Alle Änderungen sind in der Datenbank gespeichert.

Wenn eine Datei noch nicht unter der Versionskontrolle von git ist wird sie als "untracked" angezeigt.

Zustandsübergänge zwischen den unterschiedlichen Zuständen anhand eines Sequenz-Diagrams:



1.3.3 Was sind typische Aktionen?

Typische Arbeitsschritte sind:

1. Repository anlegen (oder clonen)
2. Dateien neu erstellen (und löschen, umbenennen, verschieben)
3. Änderungen einpflegen ("committen")
4. Änderungen und Logs betrachten
5. Änderungen rückgängig machen
6. Projektstand markieren ("taggen")
7. Entwicklungszweige anlegen ("branchen")

8. Entwicklungszweige zusammenführen ("mergen")
9. Änderungen verteilen (verteiltes Arbeiten, Workflows)

1.3.4 Lasst uns Spielen!

Prüfe ob du schon git hast:

1. Öffne eine Kommandozeile
2. Führe `git --version` aus

Download git, wenn `git` noch nicht installiert ist:

Linux:

```
sudo apt install git-all
```

Mac:

mit dem ausführen des Befehls `git --version` wird git automatisch installiert.

Windows:

Download via <https://git-scm.com/download/win>

Minimale Konfiguration, um kenntlich zu machen, wer die Änderung vorgenommen hat:

```
git config --global user.name <name>  
git config --global user.email <email>
```



Wir spielen ein Spiel

Download the game: <https://ohmygit.org>

oder <https://learngitbranching.js.org>

1.4 semantic versioning (<https://semver.org>)

Semantic Versioning ist ein kleiner Satz von Regeln, die es ermöglichen mit einem Blick auf die Versionsnummer eines Programmes oder eines Paketes oder Bibliothek festzustellen, welche Änderungen vorgenommen werden.

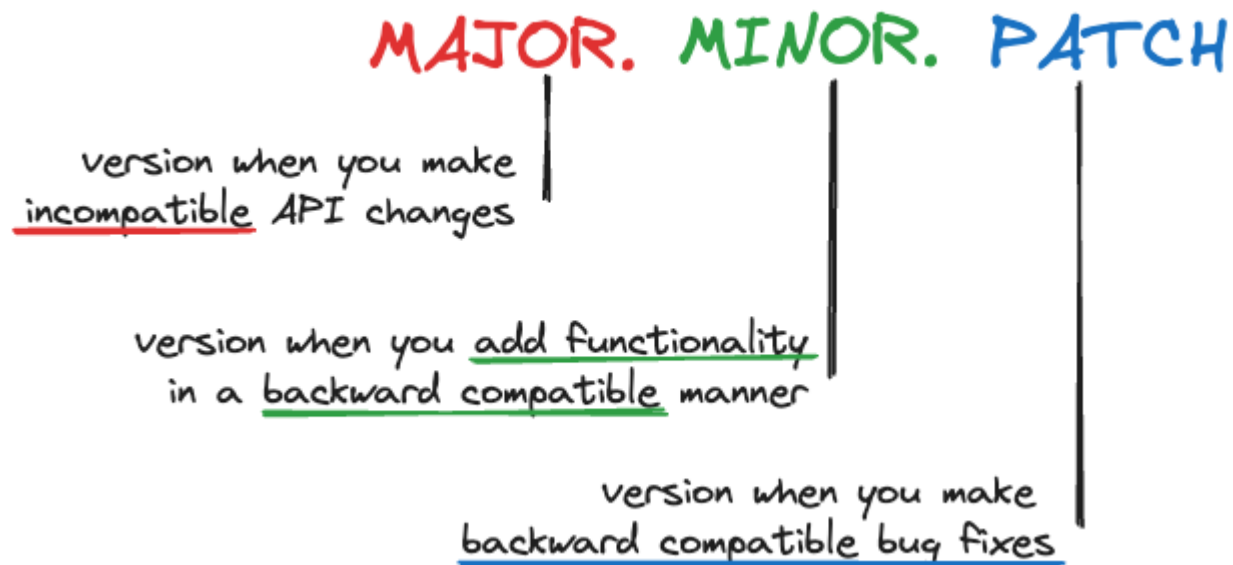
Ohne diese Regeln ist es fast unmöglich zu entscheiden, ob man ohne den Verlust von Funktionalität eine neue Version eines Paketes verwenden darf.

Definition des Änderungsgrads der Versionsnummer und Vorgehen

Bei einer Versionsnummer MAJOR.MINOR.PATCH erhöhen Sie die:

- MAJOR-Version, wenn Sie inkompatible API-Änderungen vornehmen
- MINOR-Version, wenn Sie Funktionalität auf rückwärtskompatible Weise hinzufügen
- PATCH-Version, wenn Sie abwärtskompatible Fehlerbehebungen vornehmen

Zusätzliche Bezeichnungen für Vorveröffentlichungs- und Build-Metadaten sind als Erweiterungen des Formats MAJOR.MINOR.PATCH verfügbar.



semantic versioning: Zusammensetzung der Versionsnummer

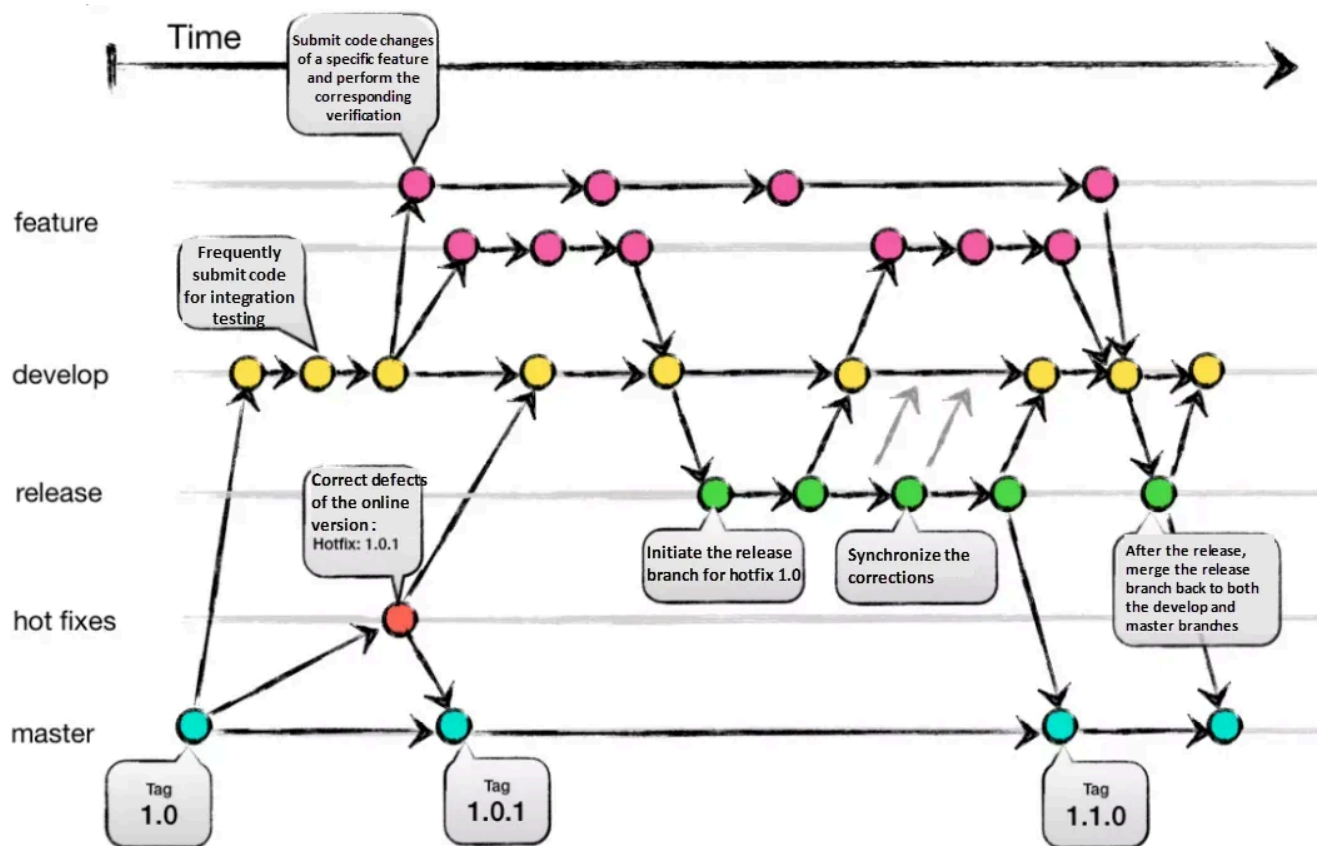
Beispiel:

Nehmen wir eine Bibliothek namens „Firetruck“. Sie benötigt ein semantisch versioniertes Paket namens „Ladder“. Zu dem Zeitpunkt, an dem Firetruck erstellt wird, hat Ladder die Version 3.1.0. Da Firetruck einige Funktionen verwendet, die erst in 3.1.0 eingeführt wurden, können Sie die Ladder-Abhängigkeit sicher als größer oder gleich 3.1.0, aber kleiner als 4.0.0 angeben. Wenn nun die Ladder-Versionen 3.1.1 und 3.2.0 verfügbar sind, können Sie sie in Ihrem Paketverwaltungssystem freigeben und wissen, dass sie mit der vorhandenen abhängigen Software kompatibel sein werden.

1.5 Branching-Strategien nach git-flow und github-flow

Es existieren unterschiedliche Strategien mit Branches die Softwareentwicklung zu strukturieren. Ein grundsätzlicher Entscheidungspunkt ist, ob man unterschiedliche Versionen seiner Software pflegen muss oder ob es immer nur eine aktive Version gibt, die auch weiterentwickelt wird.

git-flow ist besonders gut geeignet für große Software-Entwicklungen in der auch unterschiedliche Versionen gepflegt werden müssen und vor dem Release umfangreiche Tests durchgeführt werden müssen.



git-flow Vorgehen

Git-Flow hat die folgende Branches:

- Feature: ist ein Zweig für Entwickler, um Features zu entwickeln. Entwicklungszweig: ist ein Zweig, der die entwickelten Funktionen sammelt.
- Release: ist ein Zweig, der für die Versionsfreigabe verantwortlich ist.
- Hotfix: ist ein Zweig, der Online-Fehler korrigiert.
- Master: ist ein Zweig, der die Baseline der letzten veröffentlichten Version speichert.

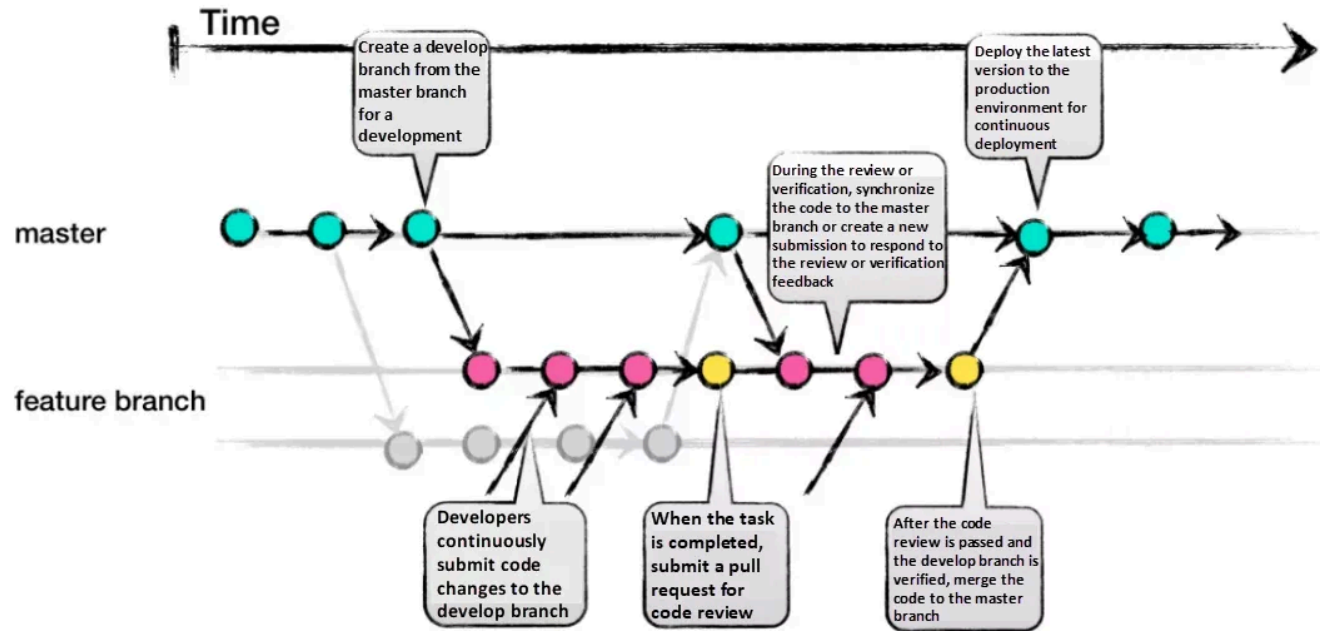
Vorteile:

- Vollständige Regeln mit eindeutigen Zuständigkeiten der Branches.
- Förderlich für die Verteilung von traditioneller Software.
- Bietet eine sehr gründliche und detaillierte Versionskontrolle, da Zusammenführungen gebündelt und klar gekennzeichnet sind und sauber nachverfolgt werden können.
- Einfache Skalierung unserer Entwicklung. Es vereinfacht die parallele Verarbeitung und die kontinuierliche Entwicklung, indem es Funktionen isoliert und sicherstellt, dass Sie weder Ihren Entwicklungs- noch Ihren Master-Zweig für die Release-Vorbereitung einfrieren müssen.
- Flexible Verzweigungsstrategie. Es ist einfacher, zwischen Teilen der Entwicklungspipeline zu springen (z. B. zwischen einer Funktion für eine spätere Version und den dringenden Prioritäten der aktuellen Version).
- Der Code im Master-Zweig bleibt sehr sauber und organisiert und wird nur mit Code aktualisiert, der gründlich getestet und verfeinert wurde.

Nachteile

- Viele Branches mit komplizierten Regeln
- Hoher Wartungsaufwand für veröffentlichte Versionen
- Die starre Struktur und der spezifische Entwicklungspfad von Git-Flow stehen im Konflikt mit dem iterativen Ansatz einer Agilen Entwicklungsmethodik.

github-flow ist eine starke Vereinfachung von der ursprünglichen git-flow Strategie, insbesondere für Software, die vor dem Deployment nicht noch separat getestet werden muss. Die Idee ist, dass bei dem Merge eines feature-branches auf den main-Branch alle Schritte durchgeführt worden sind, so dass die Software direkt danach deployed werden kann.



github-flow

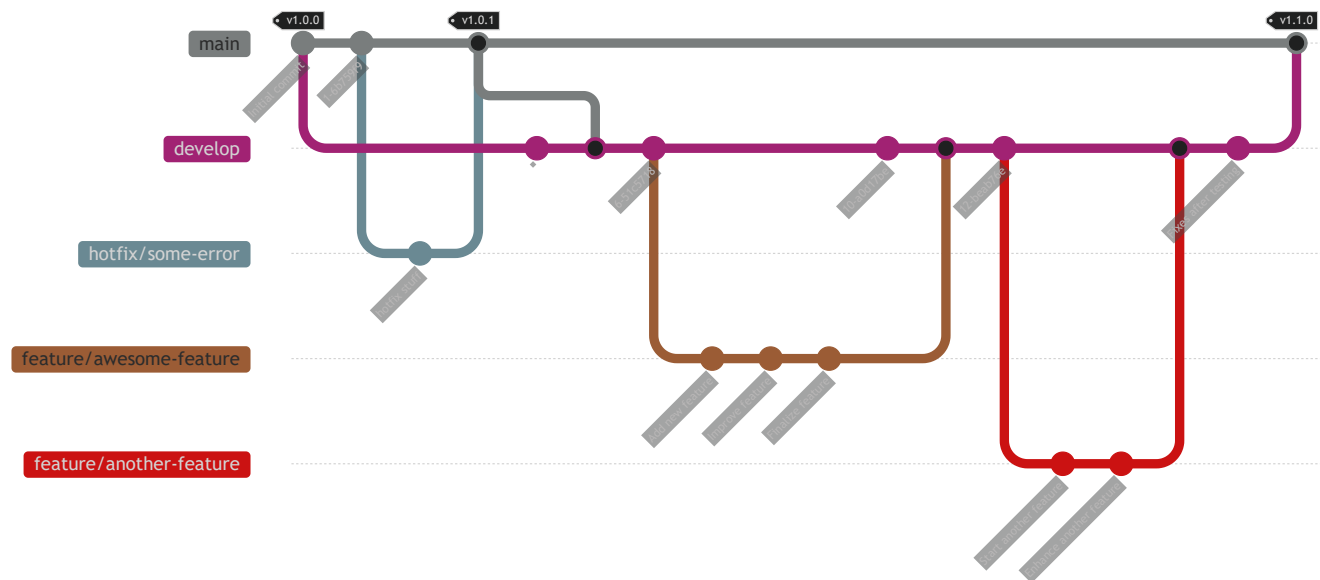
Vorteile

- Klare und einfache Regeln für die Zusammenarbeit.
- Kontinuierliche Integration und Bereitstellung.
- Fördert schnelle Feedbackschleifen und ermöglicht es Teams, Probleme schnell zu erkennen und Änderungen vorzunehmen.
- Kontinuierliche Bereitstellung ist bei GitHub Flow so gut wie ein Muss. Es gibt keinen Entwicklungsweig, in dem fertige Funktionen auf die Veröffentlichung warten. Aus dem → Erschaffenen wird schnell der Wert, der geliefert wird.
- Mit dieser Verzweigungsstrategie ist das Risiko technischer Schulden geringer. In Git-Flow kann eine Abkürzung oder ein suboptimaler Teil des Codes schnell verschüttet werden und zu einem Alptraum für das Refactoring werden. Da GitHub-Flow den Schwerpunkt auf flache Strukturen und kleine Änderungen legt, ist es auf lange Sicht einfacher, technische Schulden zu erkennen und zu verwalten.

Nachteile

- GitHub Flow ist nicht so gut organisiert wie Git-Flow. Seine Schnelligkeit geht auf Kosten der Übersichtlichkeit und kann die Verwaltung des gesamten Entwicklungsprozesses erschweren.
- Diese Verzweigungsstrategie legt den Schwerpunkt auf eine konstante Bereitstellung. Während dies für einige Softwareteams gut funktionieren kann, wird es für andere zu einer Einschränkung, wenn sie größere Veröffentlichungen anstreben oder mehrere Funktionen vor der Bereitstellung gemeinsam testen wollen.
- Der Master-Zweig kann leichter unübersichtlich werden, da er sowohl als Produktions- als auch als Entwicklungszweig fungiert. Die Vorbereitung der Veröffentlichung und die Fehlerbehebung finden beide in diesem Zweig statt - und erfordern besondere Aufmerksamkeit.

Praktischer Ansatz aus beiden Strategien



Praktischer Ansatz aus beiden Strategien

gitGraph

commit id: "Initial commit" tag: "v1.0.0"

branch develop

checkout main

commit id: ""

```
branch hotfix/some-error
checkout hotfix/some-error
commit id: "hotfix stuff"
checkout main
merge hotfix/some-error tag: "v1.0.1"
```

```
checkout develop
commit id: " "
merge main
```

```
commit id: ""
```

```
branch feature/awesome-feature
checkout feature/awesome-feature
commit id: "Add new feature"
commit id: "Improve feature"
commit id: "Finalize feature"
checkout develop
commit id: ""
merge feature/awesome-feature
commit id: ""
```

```
branch feature/another-feature
checkout feature/another-feature
commit id: "Start another feature"
commit id: "Enhance another feature"
checkout develop
merge feature/another-feature
commit id: "Fixes after testing"
```

```
checkout main
```

```
merge develop tag: "v1.1.0"
```

1.6 Git Cheat Sheet

Eine Übersicht von nützlichen git Kommandos:

Action	Command
Initialize a repository	<code>git init</code>
Clone a repository	<code>git clone <repository-url></code>
Check status of working directory	<code>git status</code>
View commit history	<code>git log</code>
View changes (diff) in working directory	<code>git diff</code>
List all branches	<code>git branch</code>
Create a new branch	<code>git branch <branch-name></code>
Switch to a branch	<code>git checkout <branch-name></code>
Create and switch to a new branch	<code>git checkout -b <branch-name></code>
Merge a branch into the current branch	<code>git merge <branch-name></code>
Delete a branch	<code>git branch -d <branch-name></code>
Stage all changes	<code>git add .</code>
Stage specific files	<code>git add <file1> <file2></code>

Action	Command
Unstage a file	<code>git reset <file></code>
Commit staged changes	<code>git commit -m "commit message"</code>
Amend the last commit	<code>git commit --amend</code>
Discard changes in a file	<code>git checkout -- <file></code>
Unstage all changes	<code>git reset</code>
Revert a commit	<code>git revert <commit-hash></code>
Reset to a previous commit	<code>git reset --hard <commit-hash></code>
Add a remote repository	<code>git remote add <name> <url></code>
List all remotes	<code>git remote -v</code>
Fetch changes from the remote	<code>git fetch</code>
Pull changes from the remote and merge	<code>git pull</code>
Push changes to the remote	<code>git push</code>
Push to a specific branch	<code>git push origin <branch-name></code>
Push all branches	<code>git push --all</code>
Delete a branch on the remote	<code>git push origin --delete <branch-name></code>
Create a new tag	<code>git tag <tag-name></code>

Action	Command
Push a tag to the remote	<code>git push origin <tag-name></code>
List all tags	<code>git tag</code>
Delete a tag	<code>git tag -d <tag-name></code>
Delete a remote tag	<code>git push origin :refs/tags/<tag-name></code>
Stash changes	<code>git stash</code>
List stashes	<code>git stash list</code>
Apply the latest stash	<code>git stash apply</code>
Apply a specific stash	<code>git stash apply stash@{<index>}</code>
Drop a specific stash	<code>git stash drop stash@{<index>}</code>
Pop (apply and remove) the latest stash	<code>git stash pop</code>
Create a new branch and track a remote branch	<code>git checkout -b <branch-name> origin/<branch-name></code>
Rebase your branch with the latest changes from the main branch	<code>git pull --rebase origin main</code>
Squash commits	<code>git rebase -i HEAD~<number-of-commits></code>
Cherry-pick a commit	<code>git cherry-pick <commit-hash></code>
Show detailed commit information	<code>git show <commit-hash></code>

Action	Command
List ignored files	<code>git status --ignored</code>
View git configuration	<code>git config --list</code>
Configure user name	<code>git config --global user.name "Your Name"</code>
Configure user email	<code>git config --global user.email "your.email@example.com"</code>
View remote branches	<code>git branch -r</code>

1.7 Code Reviews

Immer wenn ein Branch gemerged wird, dann sollte ein Code Review erfolgen, so dass sichergestellt ist, dass grundsätzliche Qualitätsanforderungen an die Software-Entwicklung erfüllt sind und darüber hinaus auch die Anforderungen an die eingesetzte Branching Strategie erfüllt sind.

Darüber hinaus sind Code Reviews eine der Möglichkeiten, um von Anderen zu lernen, wie gute Software Entwicklung erfolgt und Feedback zu seinem eigenen Code zu bekommen.

Eine Codeüberprüfung erfolgt in folgenden Schritten:

1. Identifizieren des zu überprüfenden Codes (z.B. Pull-Request auf github)
2. Vorbereiten des Codes für Änderungen (z.B. Ausfüllen eines Templates zu dem Pull-Request)
3. Auswahl der Prüfer/Reviewer
4. Durchführen der Code-Review
5. Diskutieren und Angehen von Problemen
6. Genehmigung oder Ablehnung des Codes

Es gibt viele gängige Techniken, um Code-Reviews durchzuführen. Hier wird sich auf eine Checklisten-basierte technik begrenzt.

1.7.1 Vorbereitung durch den Software Entwickler³

Als Entwickler sollte man einen Pull-Request mit verschiedenen Informationen anreichern, damit die Reviewer die Änderung besser verstehen:

English Version - Pull-Request Template

Pull-Request Template

Erläuterung der Änderung

- den Grund, warum diese Änderung notwendig ist
- warum Sie diesen Lösungsansatz gewählt haben
- welche anderen Lösungsansätze Sie in Betracht gezogen haben und warum Sie diese nicht umgesetzt haben

Informationen über die Qualität der Änderung

- werden Tests hinzugefügt (die Antwort sollte ja lauten)
- Bestehen alle Tests (die Antwort sollte auf jeden Fall ja lauten)
- bricht der Build ab (auch hier gilt: Nein ist keine Option ;-)
- gibt es Linting- oder Kompilierfehler (nein ist das, was wir hören wollen)

Anleitung für den Reviewer

- worauf sollten die Prüfer achten
- wo sollten sie mit der Überprüfung beginnen
- Wo liegt die Hauptänderung/das Hauptrisiko?

1.7.2 Checkliste für ein Code Review⁴

Code Review Checklist 1/2

Implementation

- ☐ Does this code change do what it is supposed to do?
- ☐ Can this solution be simplified?
- ☐ Does this change add unwanted compile-time or run-time dependencies?
- ☐ Was a framework, API, library, service used that should not be used?
- ☐ Was a framework, API, library, service not used that could improve the solution?
- ☐ Is the code at the right abstraction level?
- ☐ Is the code modular enough?
- ☐ Would you have solved the problem in a different way that is substantially better in terms of the code's maintainability, readability, performance, security?
- ☐ Does similar functionality already exist in the codebase? If so, why isn't this functionality reused?
- ☐ Are there any best practices, design patterns or language-specific patterns that could substantially improve this code?
- ☐ Does this code follow Object-Oriented Analysis and Design Principles, like the Single Responsibility Principle, Open-Close Principle, Liskov Substitution Principle, Interface Segregation, Dependency Injection?

Logic Errors and Bugs

- ☐ Can you think of any use case in which the code does not behave as intended?
- ☐ Can you think of any inputs or external events that could break the code?

Error Handling and Logging

- ☐ Is error handling done the correct way?
- ☐ Should any logging or debugging information be added or removed?
- ☐ Are error messages user-friendly?
- ☐ Are there enough log events and are they written in a way that allows for easy debugging?

Dependencies

- ☐ If this change requires updates outside of the code, like updating the documentation, configuration, readme files, was this done?
- ☐ Might this change have any ramifications for other parts of the system, or backward compatibility?

Security and Data Privacy

- ☐ Does this code open the software up for security vulnerabilities?
- ☐ Are authorization and authentication handled in the right way?
- ☐ Is sensitive data like user data, credit card information securely handled and stored?
- ☐ Is the right encryption used?
- ☐ Does this code change reveal some secret information like keys, passwords, or usernames?
- ☐ If code deals with user input, does it address security vulnerabilities such as cross-site scripting, SQL injection, does it do input sanitization and validation?
- ☐ Is data retrieved from external APIs or libraries checked accordingly?

Performance

- ☐ Do you think this code change will impact system performance in a negative way?
- ☐ Do you see any potential to improve the performance of the code?

Usability and Accessibility

- ☐ Is the proposed solution well designed from a usability perspective?
- ☐ Is the API well documented?
- ☐ Is the proposed solution (UI) accessible?
- ☐ Is the API/UI intuitive to use?

Code Review Checklist 2/2

Readability

- ☐ Was the code easy to understand?
- ☐ Which parts were confusing to you and why?
- ☐ Can the readability of the code be improved by smaller methods?
- ☐ Can the readability of the code be improved by different function/method or variable names?
- ☐ Is the code located in the right file/folder/package?
- ☐ Do you think certain methods should be restructured to have a more intuitive control flow?
- ☐ Is the data flow understandable?
- ☐ Are there redundant comments?
- ☐ Could some comments convey the message better?
- ☐ Would more comments make the code more understandable?
- ☐ Could some comments be removed by making the code itself more readable?
- ☐ Is there any commented out code?

Testing and Testability

- ☐ Is the code testable?
- ☐ Does it have enough automated tests (unit/integration/system tests)?
- ☐ Do the existing tests reasonably cover the code change?
- ☐ Are there some test cases, input or edge cases that should be tested in addition?

Experts Opinion

- ☐ Do you think a specific expert, like a security expert or a usability expert, should look over the code before it can be committed?
- ☐ Will this code change impact different teams? Should they have a say on the change as well?

Exercise

- Which parts of the checklist are you already considering?
- Which aspect aren't you focusing on during code reviews and why?
- Do you think some aspects are more important than others? Why? Why not?
- Do you feel you would benefit from additional training in some areas (e.g., security, accessibility)?

Notes:

Find more at michaelagreiter.com

- Code Review Best Practices
- Code Review Pitfalls
- Code Reviews at Microsoft
- Code Reviews at Google

www.michaelagreiter.com

www.michaelagreiter.com

Code Review Cecklist

Download: [Code Review Cecklist](#)

1.7.3 Leitlinien für ein Code Review ⁵

Leitlinien zum Review:

English Version

Aspekt der Überprüfung	Gute Überprüfung	Bessere Überprüfung
Abgedeckte Bereiche der Überprüfung	Konzentriert sich auf die eigentliche Code-Änderung, stellt Klarheit, Korrektheit, Testabdeckung und Einhaltung der Best Practices sicher. Beispiel: Der Prüfer überprüft, ob die neue Funktion den Namenskonventionen entspricht und ausreichend Unit-Tests enthält.	Betrachtet die Änderung im Kontext des größeren Systems, der Wartbarkeit und der potenziellen Auswirkungen auf die gesamte Architektur. Beispiel: Der Prüfer fragt, ob eine neu eingeführte Funktion besser in einer Utility-Klasse platziert werden sollte, um Redundanzen im Code zu vermeiden, und schlägt vor, komplexe Logik zu vereinfachen.
Ton der Überprüfung	Stellt offene Fragen, vermeidet starke Meinungen und bietet Alternativen an, ohne darauf zu bestehen. Beispiel: "Könntest du erklären, warum dieser Ansatz gewählt wurde? Vielleicht könnten wir auch Methode X als Alternative in Betracht ziehen."	Fügt Empathie hinzu, erkennt den Aufwand an und hebt positive Aspekte hervor, um ein unterstützenderes Umfeld zu schaffen. Beispiel: "Diese Lösung ist wirklich clever! Ich habe bemerkt, dass sie leicht vereinfacht werden könnte – was hältst du davon, Methode Y zu verwenden, um das gleiche Ergebnis zu erzielen?"
Genehmigen vs. Änderungen anfordern	Klare Unterscheidung zwischen blockierenden und nicht blockierenden Kommentaren; explizit bei der Genehmigung oder Anforderung von Änderungen. Beispiel: "Genehmigt, aber bitte erwäge, die Schleife in der nächsten Iteration zu überarbeiten. Nicht	Flexibel in der Praxis, erlaubt Folgemaßnahmen und schnellere Überprüfungen bei dringenden Angelegenheiten, während die Prinzipien fest beibehalten werden. Beispiel: "Ich verstehe, dass diese Änderung dringend ist – lass uns sie jetzt genehmigen, und ich helfe dir morgen mit einem Folge-PR,

Aspekt der Überprüfung	Gute Überprüfung	Bessere Überprüfung
	blockierend.“ oder “LGTM, aber lass uns noch ein paar Kommentare hinzufügen, um die Logik zu klären.“	um die verbleibenden Probleme zu lösen.“
Von Code-Überprüfungen zum persönlichen Gespräch	Hinterlässt Kommentare nach Bedarf, schlägt jedoch persönliche Gespräche vor, wenn die Diskussion langwierig wird. Beispiel: “Es gibt hier viele Kommentare – vielleicht sollten wir uns kurz unterhalten, um Missverständnisse zu klären, bevor wir weitermachen.“	Nimmt proaktiv Kontakt auf, wenn es viele Kommentare gibt, und spart so Zeit und Missverständnisse, indem man direkt spricht. Beispiel: “Ich habe bemerkt, dass es mehrere Punkte gibt, die geklärt werden müssen – lass uns diese bei einem kurzen Gespräch besprechen, bevor du Änderungen vornimmst.“
Kleinigkeiten (Nitpicks)	Markiert unbedeutende Kommentare klar und vermeidet es, die Überprüfung mit Kleinigkeiten zu überladen. Beispiel: “nit: Erwäge, die Reihenfolge dieser Importe anzupassen, um konsistent mit unserem üblichen Stil zu sein, aber es ist nicht kritisch.“	Erkennt, dass häufige Kleinigkeiten ein Zeichen für fehlende Standards oder Werkzeuge sind, und versucht, diese Probleme außerhalb des Code-Überprüfungsprozesses zu lösen. Beispiel: “Ich habe bemerkt, dass wir ständig Kommentare zur Import-Reihenfolge machen – wie wäre es, wenn wir einen Linter einrichten, um dies automatisch zu handhaben und diese Kleinigkeiten zu vermeiden?“
Code-Überprüfungen für neue Mitarbeiter	Hält die gleiche Qualitätsanforderung für alle aufrecht, verwendet jedoch einen	Schenkt neuen Mitarbeitern besondere Aufmerksamkeit, erklärt alternative Ansätze, verweist auf Anleitungen und ist

Aspekt der Überprüfung	Gute Überprüfung	Bessere Überprüfung
	freundlichen Ton und nimmt Kontakt auf, wenn es viele Kommentare gibt. Beispiel: "Das ist ein solider erster Beitrag! Es sind nur ein paar kleine Änderungen nötig – ich helfe dir gerne weiter, wenn du Fragen hast."	besonders positiv, um eine einladende Erfahrung zu schaffen. Beispiel: "Toller Start! Ich habe ein paar Bereiche bemerkt, die nicht ganz unseren Richtlinien entsprechen – hier ist ein Link zu unserem Style-Guide, und ich gehe ihn gerne mit dir durch."
Überprüfungen über Büros und Zeitzonen hinweg	Versucht, die Überprüfung während der Überschneidung der Arbeitszeiten durchzuführen, und bietet bei vielen Kommentaren an, direkt zu sprechen oder zu telefonieren. Beispiel: "Ich habe ein paar Kommentare hinterlassen – wenn du morgen während unserer Überschneidung verfügbar bist, lass uns diese in einem kurzen Gespräch besprechen."	Erkennt systemische Probleme mit häufigen Überprüfungen über Zeitzonen hinweg und sucht nach langfristigen Lösungen. Beispiel: "Wir stoßen immer wieder auf Verzögerungen durch Zeitzeitenunterschiede – vielleicht sollten wir darüber nachdenken, dieses Modul so zu refaktorisieren, dass die Änderungen unabhängig von jedem Team verwaltet werden können, um die Abhängigkeit von Überprüfungen über Zeitzonen hinweg zu verringern."
Unterstützung durch die Organisation	Stellt sicher, dass alle Ingenieure an Code-Überprüfungen teilnehmen, ermutigt dazu, die Qualitätsanforderung zu erhöhen, und integriert Überprüfungen in die Jobkompetenzen. Beispiel: "Code-Überprüfungen sind Teil	Etabliert strenge Regeln, wie z. B. dass kein Code ohne Überprüfung in die Produktion gelangt, investiert in Werkzeuge und Prozessverbesserungen und ermächtigt Ingenieure, den Überprüfungsprozess kontinuierlich zu verbessern. Beispiel: „Unsere Richtlinie ist, dass aller Code vor der Produktion

Aspekt der Überprüfung	Gute Überprüfung	Bessere Überprüfung
	unserer technischen Kompetenzen – lass uns bei deinem nächsten Performance-Check-in besprechen, wie du zu Überprüfungen beigetragen hast.“	überprüft wird.“ Lass uns auch automatisierte Schritte für Überprüfungen erkunden, um die Effizienz zu steigern.“

Verweise

Charcon, Scott und Ben Straub. 2024. *Pro Git* . 2. Serie. APress .

Fußnoten

1. Charcon und Straub (2024) [↗](#)
2. [Glossar BSI](#) [↗](#)
3. vgl. <https://www.awesomecodereviews.com> [↗](#)
4. vgl. <https://www.michaelagreiler.com> [↗](#)
5. [Stapelüberlauf](#) [↗](#)